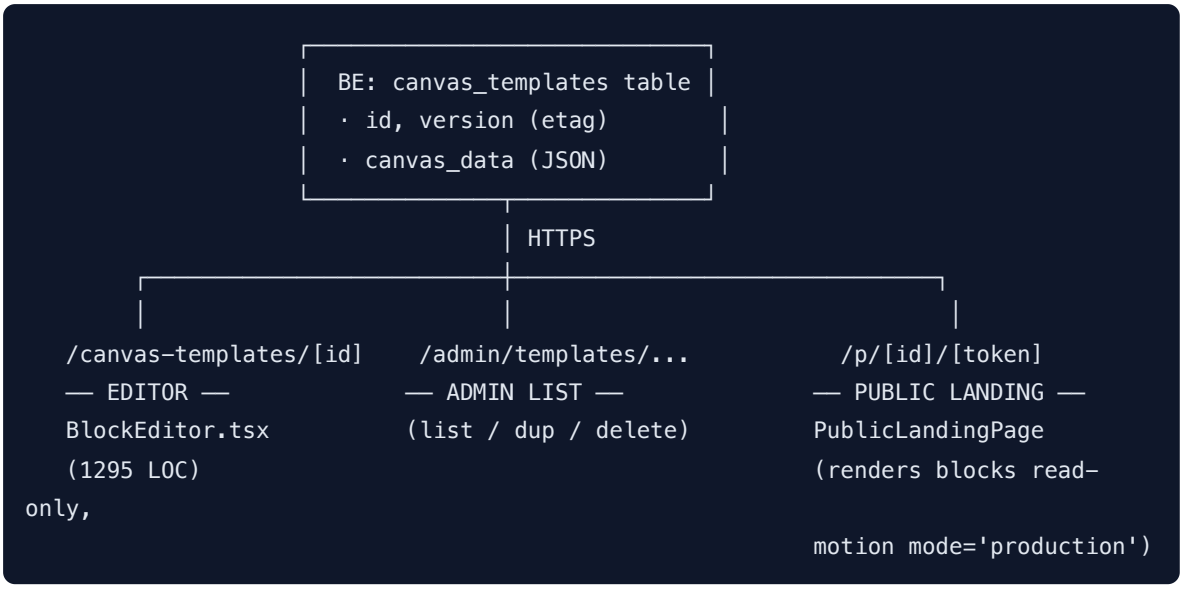
Canvas Template Architecture

Kiến trúc xử lý 1 canvas template

1. Cấu trúc tổng thể

Một **template** = 1 record trong BE (`CanvasTemplate`) + 1 trường JSON `canvas_data` chứa toàn bộ blocks. FE có 3 mặt phẳng tiêu thụ template:



3 mặt phẳng dùng CHUNG:

- `parseCanvas` / `serializeCanvas` (`block-editor/serialize.ts`)
- `RenderBlock` switcher (`blocks/renderBlock.tsx`)
- 6 block components (`blocks/HeroSplit.tsx` , ...)
- Motion runtime (`motion/runtime.tsx`)

---

2. Data model

Block (`block-editor/types.ts:170`)

```
interface Block {
  id: string;                                  // 'b_<8-hex>'
  blockType: BlockType;                        // discriminator → switch trong
      RenderBlock
  name: string;                                // hiển thị ở tab bar
  isVisible: boolean;                          // ẩn ↔ skip lúc render public
  thumbnail?: string;

  contentVi: DeviceVariants<BlockContent>;     // i18n (vi)
  contentEn: DeviceVariants<BlockContent>;     // i18n (en)

  elementStyles?: DeviceVariants<ElementStylesMap>;  // per-element CSS overrides
  background?: DeviceVariants<BlockBackground>;      // block-level bg color/image
  elementOrder?: DeviceVariants<string[]>;           // CSS `order:N` reorder
  motion?: DeviceVariants<BlockMotionStored>;        // GSAP timeline config
}
```

Variant model (`types.ts:140`)

```
interface DeviceVariants<T> { desktop: T; mobile?: T; }

readVariant<T>(v, device): mobile → fallback desktop, tablet → desktop
editingVariantOf(device): 'desktop' | 'mobile'  // tablet → desktop (read-only)
```

Quy tắc: desktop **bắt buộc**, mobile optional. Khi user chưa custom mobile, renderer dùng desktop. Tablet không bao giờ có variant riêng — luôn render desktop layout.

BlockContent (discriminated union)

Mỗi blockType có 1 interface content riêng (`HeroSplitContent` , `HeroOverlayContent` , ...). Field tên trùng với `data-element-key` trên DOM (vd: `title` , `eyebrow` , `imageUrl`). Đây là invariant quan trọng — motion system, element style, AI apply-product, element toolbar đều join giữa content và DOM qua tên key này.

---

3. Wire format & migration (`serialize.ts`)

Out (serializeCanvas)

```
serializeCanvas(blocks, activeBlockId, publish) → {
  schema: 'block_editor',     // V1 legacy field
  version: 2,                 // V1 legacy field
  schema_version: '2.0',      // V2 current (BE prefer field)
  blocks: [...],              // camelCase, không transform
  active_block_id?,
  publish?: { published, shareToken, publishedAt }
}
```

Quan trọng: chỉ top-level meta là snake_case. Inner block fields **camelCase** (BE validator yêu cầu). Một regression trước đây deep-converted → BE reject với “missing blocks[0].contentVi”.

In (parseCanvas)

1. Detect schema: `schema_version === '2.0'` ưu tiên, fallback `schema === 'block_editor'` .
2. Mỗi block đi qua `migrateBlockV1toV2` :
   - Flat content → wrap thành `{ desktop: ... }` .
   - Migration motion từ format preset cũ → mới track-based: **bỏ** preset cũ (không có mapping). Tracks payload đi thẳng qua.
3. Trả về `BlockEditorCanvas` .

---

4. Editor (`BlockEditor.tsx`)

Top-level state

```
const [template, setTemplate]                 // CanvasTemplate meta (etag = version)
const [blocks, setBlocks]                     // Block[] — single source of truth
const [activeId, setActiveId]                 // ID block đang active
const [lang, setLang]                         // 'vi' | 'en'
const [device, setDevice]                     // desktop/tablet/mobile
const [zoom, setZoom]                         // 20..100%
const [dirty, setDirty]                       // có thay đổi chưa lưu?
const [saving, setSaving]
const [saveBlocked, setSaveBlocked]           // pause autosave sau 1 lỗi
const [publish, setPublish]                   // PublishMeta
const [selectedElementKey, setSelectedElementKey]
const [sidebarTab, setSidebarTab]             // 'layers' | 'motion'
const [motionPreviewing, ...]                 // 5s preview window
const [motionPreviewNonce, ...]               // re-trigger play
const [motionPreviewElement, ...]             // audition 1 element only
const [reviewState, setReviewState]           // AI apply-product diff dialog
const [previewing, setPreviewing]             // PreviewMode (full-screen)

const undoStackRef  = useRef<Block[][]>([])  // max 50 snapshots
const redoStackRef  = useRef<Block[][]>([])
```

Ref-mirror pattern

Mỗi state quan trọng (template, blocks, activeId, dirty, saving) có 1 `ref.current` mirror cập nhật qua `useEffect`. Lý do: `doSave` callback dùng `useCallback([])` (no deps) để stable identity cho `addEventListener('beforeunload')`, nên đọc state qua refs thay vì closure-stale state.

Lifecycle

```
mount
 ├─ fetch CanvasTemplate meta (etag = version)
 ├─ fetch canvas_data (JSON)
 ├─ parseCanvas → blocks[] (fallback seedBlocks() nếu fail/empty)
 └─ setActiveId(parsed.activeBlockId ?? blocks[0].id)

user mutates content/style/motion
 ├─ pushHistory() → undoStack.push(structuredClone(blocks))
 ├─ setBlocks(prev => prev.map(...))
 └─ setDirty(true)

debounced autosave (AUTOSAVE_DELAY_MS = 1500ms)
 ├─ chờ dirty + !saving + !saveBlocked
 ├─ serializeCanvas → PATCH /canvas-templates/{id}
 │      body: { canvas_data, version: template.version }   // optimistic
concurrency
 ├─ on 409 conflict → toast "đã bị sửa ở chỗ khác"
 ├─ on 4xx validation → log debug payload, set saveBlocked=true (chặn retry loop)
 └─ setDirty(false), update template.version từ response

beforeunload + unmount
 └─ flush save (best-effort, không async)
```

Mutation pipeline (mọi handler tuân thủ pattern)

```
function handleXxx(...) {
  if (!activeBlock) return;
  if (device === 'tablet') { toast.info('tablet read-only'); return; }
  pushHistory();                        // 1. Snapshot trước thay đổi
  setBlocks(prev => prev.map(b => {     // 2. Functional update
    if (b.id !== activeBlock.id) return b;
    return { ...b, /* edits */ };
  }));
  setDirty(true);                       // 3. Trigger autosave
}
```

History stack: simple `Block[][]` với cap 50. `redoStack` reset mỗi lần `pushHistory` (chuẩn linear-undo). Force re-render qua `useReducer` để cập nhật canUndo/canRedo trên header.

---

5. Per-block render pipeline (`RenderBlock`)

```
BlockEditor / PublicLandingPage / PreviewMode / MotionTimelineModal
        |
        | (pass block + resolved content/styles/background/order)
        ▼
    ┌──────────────────────────┐
    | <MotionProvider mode=…> |  motion context (BlockMotion)
    | <MotionBlockRoot>       |  GSAP timeline driver
    |   <RenderBlock>         |  switch(blockType)
    |     <HeroSplit /> | …   |
    └──────────────────────────┘
```

`RenderBlock` (`blocks/renderBlock.tsx:36`) là 1 switch trên `block.blockType` → dispatch sang component cụ thể. Resolved content (đã `readVariant`) được pass xuống — block component không biết gì về DeviceVariants.

Block component shape

```
function HeroSplit({ content, styles, background, order, activeKey }) {
  // 1. Compute per-element inline style merge:
  //    - elementStyleOf(styles, key) → color/font/size/etc
  //    - orderStyle(order, key, ...) → adds `order: N` for CSS flexbox reorder
  const sTitle = orderStyle(order, 'title', elementStyleOf(styles, 'title'));
  // ...

  return (
    <div style={blockBackgroundStyle(background)}      // bg color/image override
         className='animate-in fade-in ... bg-neutral-950'>  // baked-in default
CSS
      <h2 data-element-key='title' style={sTitle}
          className={selectableClass(activeKey==='title', 'animate-reveal-x ...')}>
        {content.title}
      </h2>
      {/* ... */}
    </div>
  );
}
```

Convention:

1. Mỗi element bên trong có `data-element-key='<key>'` matching field name của content. Đây là contract dùng bởi: motion runtime (querySelector), element click selection (BlockPreview's click handler walks `.closest('[data-element-key]')`), inline editor.
2. `selectableClass(active, base)` thêm outline cyan khi element đang selected — chỉ visual cho editor; production CSS unused.

3. `bg-neutral-*` default trên root được override bằng inline `style={blockBackgroundStyle(background)}` (specificity inline > Tailwind class).

Per-element render helpers (`blocks/applyStyle.ts`)

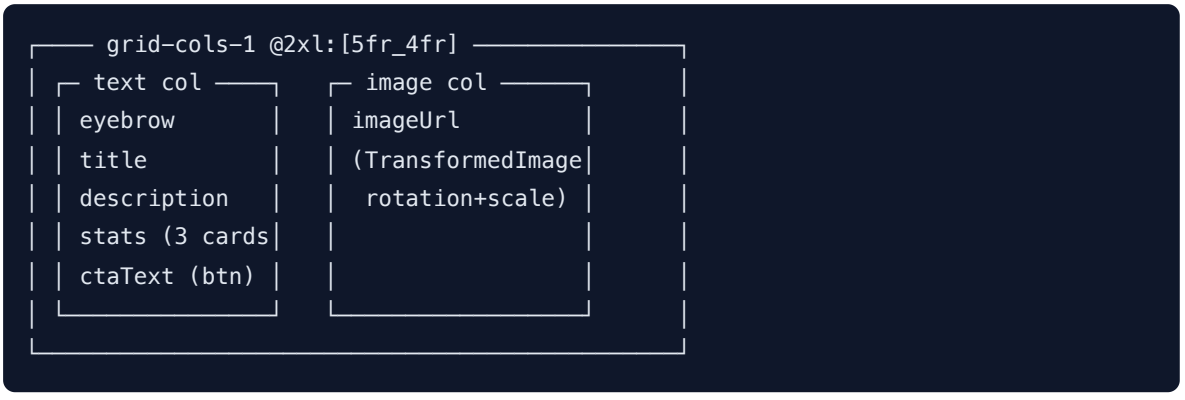
Helper	Trả về	Mục đích
`elementStyleOf(styles, key)`	`React.CSSProperties | undefined`	Pure subset → inline style cho color/font/etc
`blockBackgroundStyle(bg)`	`React.CSSProperties | undefined`	Build inline `background-color` / `background-image` (cover/center)
`orderStyle(order, key, extra)`	`React.CSSProperties | undefined`	Inline `order: N` cho CSS flexbox; merge với extra
`selectableClass(active, base)`	string	Cursor pointer + outline cyan khi active

`undefined` fallback khắp nơi → khi user chưa override, block dùng nguyên Tailwind class mặc định.

---

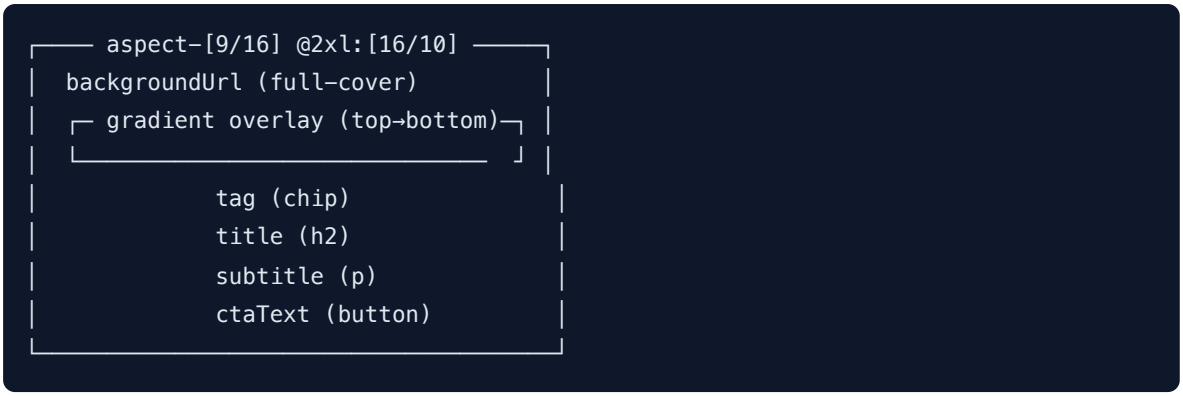
6. Mỗi block type chi tiết

6.1 HeroSplit (`HeroSplit.tsx`)



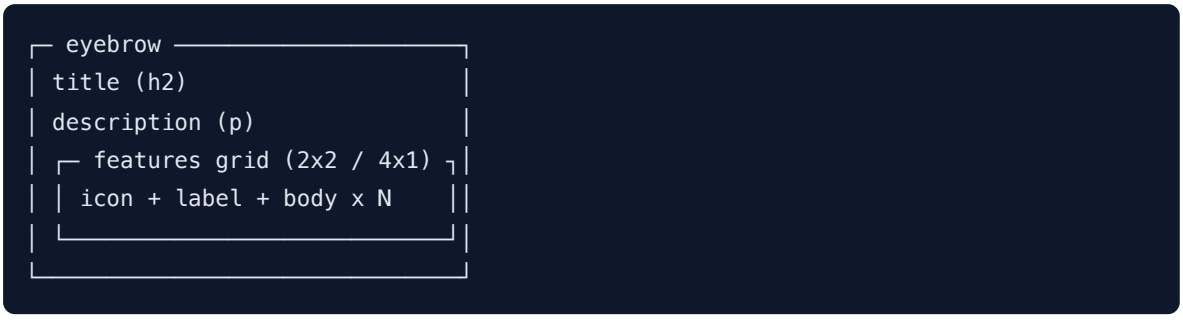
- Element keys: `eyebrow`, `title`, `description`, `stats`, `ctaText`, `imageUrl`.
- Stats = array `{value, label}[]` (typical 3 items).
- Tailwind container queries `@2xl:` (~640px) split layout từ 1-col → 2-col.
- CSS animations baked-in: `animate-reveal-x` (clip-path + opacity, eyebrow 900ms / title 1100ms), `animate-soft-rise` (translateY+opacity, description 1700ms / stats 2200ms / cta 2400ms). Runtime motion vô hiệu bằng `style.animation='none'` trên track elements.

6.2 HeroOverlay (`HeroOverlay.tsx`)



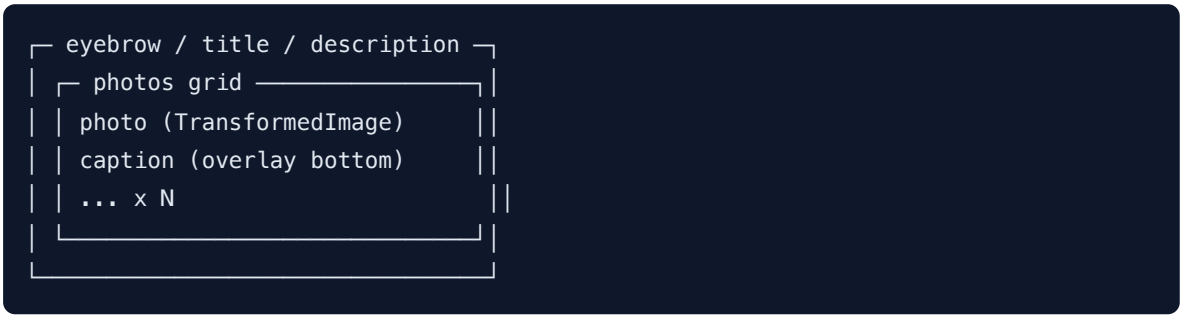
- Element keys: `tag`, `title`, `subtitle`, `ctaText`, `backgroundUrl`.
- Background image với `bg-gradient-to-t from-black/75` để text trắng dễ đọc.
- Aspect ratio cứng theo container query (mobile portrait, desktop landscape).

6.3 FeatureSection (`FeatureSection.tsx`)



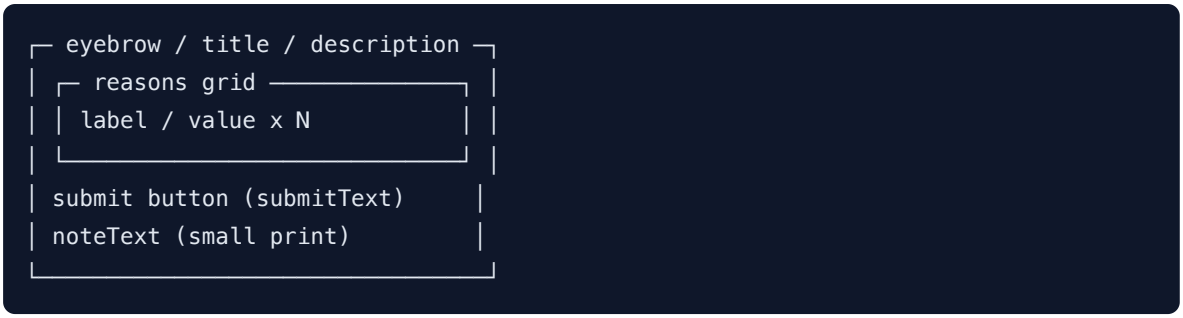
- Element keys: `eyebrow`, `title`, `description`, `features`.
- Features = `{icon, label, body}[]` — `icon` là tên Lucide icon string, render qua `LucideIcon` lookup.
- Grid responsive: 1 col mobile → 2 col `@2xl` → 4 col `@4xl`.

6.4 PhotoGallery (`PhotoGallery.tsx`)



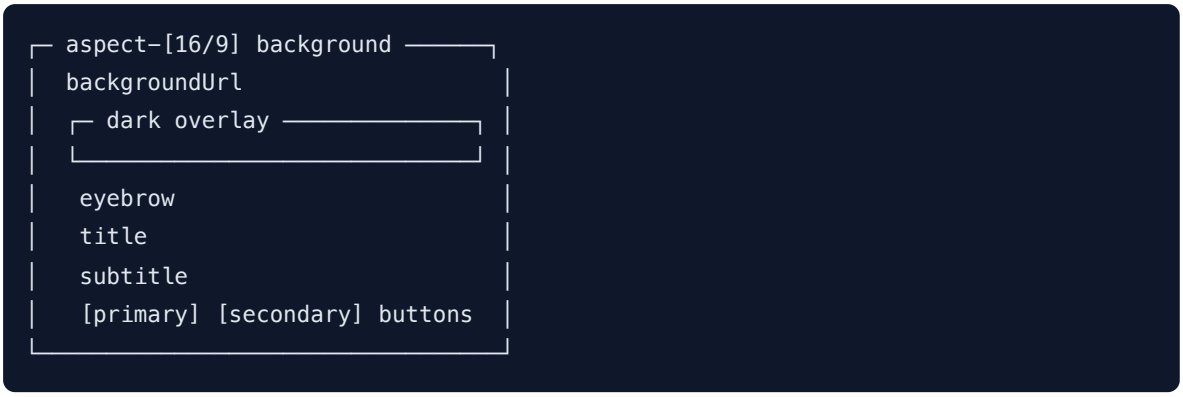
- Element keys: `eyebrow`, `title`, `description`, `photos`.
- Photos = `{url, caption, transform?}[]` — transform per-photo cho rotation/scale.

6.5 BookingForm (`BookingForm.tsx`)



- Element keys: `eyebrow`, `title`, `description`, `reasons`, `submitText`, `noteText`.
- Reasons = `{label, value}[]` — kê các lý do user nên đặt chỗ.
- **Không** có logic submit thật ở public — chỉ visual.

6.6 CtaSection (`CtaSection.tsx`)



- Element keys: `eyebrow`, `title`, `subtitle`, `primaryText`, `secondaryText`, `backgroundUrl`.
- Có 2 CTA (primary + secondary) → mỗi cái có text + link riêng.

---

7. Motion system (`block-editor/motion/`)

```
types.ts            BlockMotion { tracks, duration, trigger, repeat, yoyo, enabled
}
                    MotionTrack { elementKey, keyframes[] }
                    MotionKeyframe { time, values, easing? }
runtime.tsx         MotionProvider (context) + MotionBlockRoot (useGSAP driver)
presets.ts          9 presets (Fade/Slide/Scale/Zoom/Blur) ↔ tracks
                    (applyPreset / detectPreset)
MotionLayersPanel   Sidebar UI — preset dropdown + delay/duration + collapse/expand
timeline/*          Advanced timeline editor (modal):
                    keyframes, scrubber, inspector
```

Runtime pipeline

```
<MotionProvider mode={editorMode} motion={resolved}
                previewElement? previewNonce?>
  <MotionBlockRoot ref>
    <RenderBlock />
  </MotionBlockRoot>
</MotionProvider>

MotionBlockRoot:
  1. useLayoutEffect: target.style.animation='none' cho mỗi tracked elementKey
     → vô hiệu CSS animation baseline (animate-soft-rise/reveal-x) trước paint
  2. useGSAP (deps: mode, motion, scrollerEl, previewNonce, previewElement,
              isScrub):
     - mode='edit': return (no animation)
     - mode='preview' (editor): tl.play(0) immediately, bỏ qua trigger
     - mode='production': respect trigger
         · load        → tl.play(0)
         · in-view     → ScrollTrigger start='center bottom', once:true
         · scroll-progress → ScrollTrigger scrub
     - scrubMs (timeline modal): tl.seek paused
```

Modes

mode	Khi nào	Behavior
`edit`	Editor mặc định	No animation. CSS animation cũng bị tắt nếu có track (useLayoutEffect).
`preview`	Editor bấm “Phát preview” 5s	Build timeline, `tl.play(0)` ngay (bỏ qua trigger).
`production`	Public landing page (`/p/[id]/[token]`)	Respect trigger. `in-view` dùng ScrollTrigger 50% visible.

Single-element audition

`previewElement: string | null` filter tracks: khi user pick preset trên panel → audition 1 element 2.5s. `playElementPreview(key)` set `selectedElementKey=key` (outline) + `motionPreviewElement=key` (filter) + nonce++ (force restart).

---

8. Public landing page (`app/p/[id]/[token]/page.tsx`)

```
export default function PublicLandingPage({ params }) {
  const { id, token } = use(params);

  // 1. Fetch unauth — BE validates share_token, returns 404 if unpublished
  const resp = await rexerApi.canvasTemplates.getPublicCanvas(id, token);
  const parsed = parseCanvas(resp?.canvas_data);

  // 2. Render visible blocks vertically, motion mode='production'
  const visible = canvas?.blocks.filter(b => b.isVisible) ?? [];

  return (
    <main className='mx-auto max-w-[1280px]'>
      {visible.map(block => {
        const content = block.contentVi.desktop;  // public = always vi/desktop
        return (
          <MotionProvider key={block.id} mode='production'
                          motion={block.motion?.desktop}>
            <MotionBlockRoot>
              <RenderBlock block={block} content={content} {...rest} />
            </MotionBlockRoot>
          </MotionProvider>
        );
      })}
    </main>
  );
}
```

Khác biệt vs editor:

- Đọc qua endpoint **public** (`getPublicCanvas(id, token)`) — không auth, BE validate token.
- **Luôn** render `desktop` variant + tiếng Việt (mobile/EN chưa được surface cho public).
- 4 status: `loading` / `ok` / `not-published` (404) / `error`.
- Không có sidebar, header, edit handlers — chỉ render + motion runtime.
- Mỗi block có `key={block.id}` trên `<MotionProvider>` → block thay đổi sẽ remount sạch.

---

9. Persistence/conflict handling

Optimistic concurrency

- BE trả `template.version` (etag). FE gửi kèm mọi PATCH.
- BE so sánh version: nếu trùng → tăng version, lưu. Nếu khác → 409 conflict.

- FE 409 → toast "đã bị sửa ở chỗ khác — reload trang".

Autosave debouncing

- `dirty` flip true → 1500ms timer → save silent.
- Mỗi mutation reset timer (clearTimeout/setTimeout trong useEffect).
- `saveBlocked` flag pause autosave sau 1 lỗi (không spam toast).
- Reset `saveBlocked=false` mỗi lần `pushHistory` (user mutate thêm).

Publish

```
PublishMeta {
  published: bool
  shareToken: string  // crypto.randomUUID 24 chars
  publishedAt: ISO
}
```

Token cố định per template — không đổi mỗi lần publish lại. URL public: `/p/{templateId}/{shareToken}`.

---

10. AI apply-product flow

```
user picks product → handleApplyProduct(product)
 ├─ snapshot originalBlocks = structuredClone(blocks)
 ├─ POST /v1/ai/canvas-templates/{id}/apply-product
 │       { product_id, lang, preserve_user_overrides, expected_canvas_etag }
 ├─ parseCanvas(response.canvas_data) → aiBlocks
 ├─ diffCanvas(originalBlocks, aiBlocks, lang) → CanvasDiff
 │       groups per block: text / image / list / cta
 ├─ setReviewState({ diff, aiBlocks, originalBlocks })
 │
 │  ReviewChangesDialog: user accept/reject từng (block, group)
 │
 ├─ mergeAcceptedChanges(original, ai, acceptMap) → finalBlocks
 ├─ setBlocks(finalBlocks); setDirty(true)
 └─ autosave kicks in

Fallback: nếu AI fail → applyProductToBlocks(prev, product)
                      (deterministic client-side mapping)
```

---

11. Element selection & inline edit

- User click trên canvas → `BlockPreview.handleClick` walks `.closest('[data-element-key]')` → `setSelectedElementKey(key)`.
- Element selected → `selectableClass(active=true)` thêm outline `outline-sky-500`.

- `ElementToolbar` (floating, dùng `rect` của element) hiện ABCDE+++ controls: color, font, alignment, etc.
- Double-click trên text element → `handleStartInlineEdit(key)` → set `editingKey`, render với `contenteditable=true`. Commit on blur/Enter qua `handleCommitInlineEdit(key, value)`.

---

12. Tóm tắt invariants

Invariant	Lý do
`data-element-key` trên DOM = field name trong `BlockContent`	Motion query, click selection, AI diff, inline edit đều join qua key
desktop variant **bắt buộc** trong DeviceVariants	Mobile falls back to desktop. Tablet đọc desktop (read-only).
`block.id` cố định kể từ khi tạo	History snapshots, motion config key, BE persistence đều dùng id
Inner block fields **camelCase** trong wire	BE validator V2.0 yêu cầu (không deep transform)
Mọi mutation gọi `pushHistory()` trước `setBlocks()`	Cho phép Ctrl+Z hoàn nguyên 50 bước
Motion `__block__` key = MotionBlockRoot div (block root)	Animate cả block thay vì element
`useLayoutEffect` set `animation:none` chạy trước paint	Tránh CSS animation flicker khi GSAP timeline cũng animate cùng property
`previewNonce` bump mỗi click Play	Force GSAP re-build vì state-equality dedup

---

13. File tree summary

```
src/
├─ app/
│  ├─ (app)/canvas-templates/[id]/page.tsx    Editor route
│  └─ p/[id]/[token]/page.tsx                 Public landing route
└─ components/modules/CanvasTemplate/
   ├─ BlockEditor.tsx                          1295 LOC — top-level orchestrator
   └─ block-editor/
      ├─ types.ts                              Block, DeviceVariants, content
      ├─ serialize.ts                          parseCanvas / serializeCanvas
      ├─ factory.ts                            createBlock(type) + defaults
      ├─ diffCanvas.ts                         AI diff per-block/per-group
      ├─ mergeAcceptedChanges.ts               Apply user-accepted AI changes
      ├─ applyProduct.ts                       Client-side fallback mapping
      ├─ BlockEditorHeader.tsx                 Top bar (lang, device, save,
publish)
      ├─ BlockEditorSidebar.tsx                Tab strip + Layers/Motion panels
      ├─ BlockPreview.tsx                      Canvas surface — selection, edit
      ├─ BlockTabBar.tsx                       Block thumbnails + reorder (dnd-kit)
      ├─ LayersPanel.tsx                       Element tree + reorder + mobile
badge
      ├─ ElementToolbar.tsx                    Float toolbar khi element selected
      ├─ AddBlockDialog.tsx                    Picker để add block mới
      ├─ ReviewChangesDialog.tsx               AI diff acceptance UI
      ├─ PreviewMode.tsx                       Full-screen preview
      ├─ PublishDialog.tsx                     Publish + share link
      ├─ DuplicateTemplateDialog.tsx
      ├─ ProductPickerDialog.tsx
      ├─ ImagePicker.tsx
      ├─ blocks/
      │  ├─ renderBlock.tsx                    Switch on blockType
      │  ├─ applyStyle.ts                      Style merge helpers
      │  ├─ TransformedImage.tsx               <img> với rotation+scale
      │  ├─ HeroSplit.tsx
      │  ├─ HeroOverlay.tsx
      │  ├─ FeatureSection.tsx
      │  ├─ PhotoGallery.tsx
      │  ├─ BookingForm.tsx
      │  └─ CtaSection.tsx
      └─ motion/
         ├─ types.ts                           BlockMotion / MotionTrack / Keyframe
         ├─ runtime.tsx                        MotionProvider + MotionBlockRoot
         ├─ presets.ts                         9 preset library
         ├─ MotionLayersPanel.tsx              Sidebar simple-mode UI
         └─ timeline/                           Advanced timeline modal
            ├─ MotionTimelineModal.tsx
            ├─ BlockControlsBar.tsx
            ├─ TimelineRuler.tsx
            ├─ TrackRow.tsx
```

```
├─ KeyframeInspector.tsx
├─ AddTrackPopover.tsx
└─ utils.ts
```